

ENV 797 - Time Series Analysis for Energy and Environment Applications | Spring 2026

Assignment 5 - Due date 02/17/26

Zhaoxin Zhang

Directions

You should open the .rmd file corresponding to this assignment on RStudio. The file is available on our class repository on Github.

Once you have the file open on your local machine the first thing you will do is rename the file such that it includes your first and last name (e.g., “LuanaLima_TSA_A05_Sp26.Rmd”). Then change “Student Name” on line 3 with your name.

Then you will start working through the assignment by **creating code and output** that answer each question. Be sure to use this assignment document. Your report should contain the answer to each question and any plots/tables you obtained (when applicable).

When you have completed the assignment, **Knit** the text and code into a single PDF file. Submit this pdf using Canvas.

R packages needed for this assignment: “readxl”, “ggplot2”, “forecast”, “tseries”, and “Kendall”. Install these packages, if you haven’t done yet. Do not forget to load them before running your script, since they are NOT default packages.\

```
#Load/install required package here
```

```
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
```

```
##   method      from
```

```
##   as.zoo.data.frame zoo
```

```
library(tseries)
```

```
library(ggplot2)
```

```
library(Kendall)
```

```
library(lubridate)
```

```
##
```

```
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##   date, intersect, setdiff, union
```

```
library(tidyverse) #load this package so you can clean the data frame using pipes
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
## v dplyr 1.2.0 v stringr 1.5.1  
## v forcats 1.0.0 v tibble 3.2.1  
## v purrr 1.0.2 v tidyr 1.3.1  
## v readr 2.1.5
```

```
## -- Conflicts ----- tidyverse_conflicts() --  
## x dplyr::filter() masks stats::filter()  
## x dplyr::lag() masks stats::lag()  
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(readxl)
```

Consider the same data you used for A04 from the spreadsheet “Table_10.1_Renewable_Energy_Production_and_Consumption”. The data comes from the US Energy Information Administration and corresponds to the December 2025 Monthly Energy Review.

```
#Importing data set - using readxl package  
energy_data <- read_excel(  
  path = "/Data/Table_10.1_Renewable_Energy_Production_and_Consumption_by_Source.xlsx",  
  skip = 12,  
  sheet = "Monthly Data",  
  col_names = FALSE  
)
```

```
## New names:  
## * ' ' -> '...1'  
## * ' ' -> '...2'  
## * ' ' -> '...3'  
## * ' ' -> '...4'  
## * ' ' -> '...5'  
## * ' ' -> '...6'  
## * ' ' -> '...7'  
## * ' ' -> '...8'  
## * ' ' -> '...9'  
## * ' ' -> '...10'  
## * ' ' -> '...11'  
## * ' ' -> '...12'  
## * ' ' -> '...13'  
## * ' ' -> '...14'
```

```
#Now let's extract the column names from row 11 only  
read_col_names <- read_excel(  
  path = "/Data/Table_10.1_Renewable_Energy_Production_and_Consumption_by_Source.xlsx",  
  skip = 10,  
  n_max = 1,  
  sheet = "Monthly Data",  
  col_names = FALSE  
)
```

```
## New names:
## * ' ' -> '...1'
## * ' ' -> '...2'
## * ' ' -> '...3'
## * ' ' -> '...4'
## * ' ' -> '...5'
## * ' ' -> '...6'
## * ' ' -> '...7'
## * ' ' -> '...8'
## * ' ' -> '...9'
## * ' ' -> '...10'
## * ' ' -> '...11'
## * ' ' -> '...12'
## * ' ' -> '...13'
## * ' ' -> '...14'
```

```
colnames(energy_data) <- read_col_names
nobs <- nrow(energy_data)

nobs=nrow(energy_data)
nvar=ncol(energy_data)

head(energy_data)
```

```
## # A tibble: 6 x 14
##   Month      'Wood Energy Production' 'Biofuels Production'
##   <dtm>                                <dbl> <chr>
## 1 1973-01-01 00:00:00                130. Not Available
## 2 1973-02-01 00:00:00                117. Not Available
## 3 1973-03-01 00:00:00                130. Not Available
## 4 1973-04-01 00:00:00                125. Not Available
## 5 1973-05-01 00:00:00                130. Not Available
## 6 1973-06-01 00:00:00                125. Not Available
## # i 11 more variables: 'Total Biomass Energy Production' <dbl>,
## #   'Total Renewable Energy Production' <dbl>,
## #   'Hydroelectric Power Consumption' <dbl>,
## #   'Geothermal Energy Consumption' <dbl>, 'Solar Energy Consumption' <chr>,
## #   'Wind Energy Consumption' <chr>, 'Wood Energy Consumption' <dbl>,
## #   'Waste Energy Consumption' <dbl>, 'Biofuels Consumption' <chr>,
## #   'Total Biomass Energy Consumption' <dbl>, ...
```

Handling Missing Data

Q1

Using the original dataset, create a new data frame that includes only the following variables: **Date**, **Solar Energy Consumption** and **Wind Energy Consumption**. Check the class of columns, you will see that they are stored as characters instead of numbers. Because solar generation begins later in the sample, the early observations are recorded as “Not Available”. Convert the data to numeric, the “Not Available” will become NAs.

You may either filter out the “Not Available” rows and then convert the column to numeric or convert first and then remove missing values using `drop_na()` (or `na.omit()`). If you are comfortable using pipes for data wrangling, please do so.

Important: Note that we dropping the missing observations instead of interpolating is because they only happen in the beginning of the series!

```
SolarWind_data <- energy_data[,c(1,8,9)] #Select only the Solar and Wind Energy Consumption
class(SolarWind_data)

## [1] "tbl_df"      "tbl"        "data.frame"

#Convert Solar Consumption into numeric
SolarWind_data$`Solar Energy Consumption` <- as.numeric(SolarWind_data$`Solar Energy Consumption`)

## Warning: NAs introduced by coercion

#Convert Wind Consumption into numeric
SolarWind_data$`Wind Energy Consumption` <- as.numeric(SolarWind_data$`Wind Energy Consumption`)

## Warning: NAs introduced by coercion

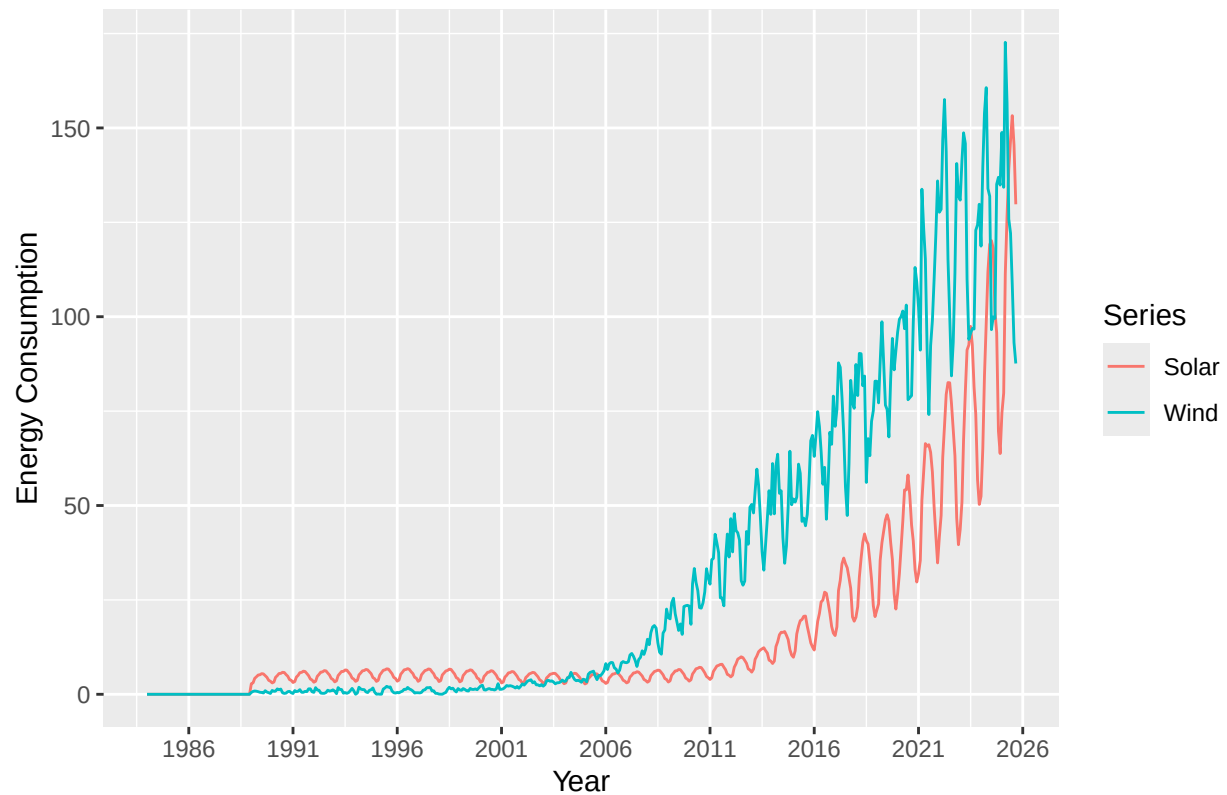
#Remove NAs
SolarWind_data <- na.omit(SolarWind_data)
#class(SolarWind_data$Month)
SolarWind_data$Month <- as.Date(SolarWind_data$Month)
#class(SolarWind_data$Month)
```

Q2

Plot the Solar and Wind energy consumption over time using ggplot. Plot each series on a separate graph. No need to add legend. Add informative names to the y axis using `ylab()`. Explore the function `scale_x_date()` on ggplot and see if you can change the x axis to improve your plot. Hint: use `scale_x_date(date_breaks = "5 years", date_labels = "%Y")`

```
ggplot(SolarWind_data, aes(x = Month)) +
  geom_line(aes(y = `Solar Energy Consumption`, color = "Solar")) +
  geom_line(aes(y = `Wind Energy Consumption`, color = "Wind")) +
  ggtitle("Solar and Wind Energy Consumption")+
  ylab("Energy Consumption")+
  xlab("Year")+
  scale_color_discrete(name = "Series") +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y")
```

Solar and Wind Energy Consumption

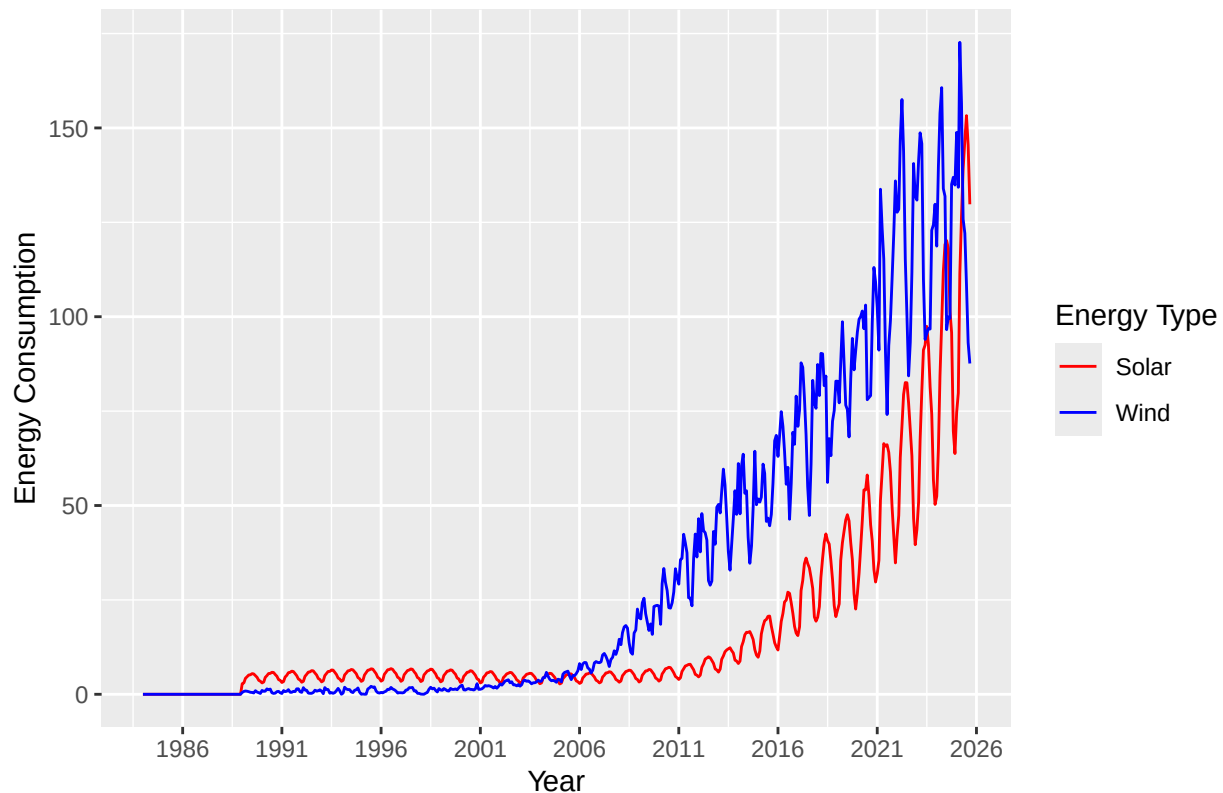


Q3

Now plot both series in the same graph, also using `ggplot()`. Use function `scale_color_manual()` to manually add a legend to `ggplot`. Make the solar energy consumption red and wind energy consumption blue. Add informative name to the y axis using `ylab("Energy Consumption")`. And use function `scale_x_date()` to set x axis breaks every 5 years.

```
ggplot(SolarWind_data, aes(x = Month)) +
  geom_line(aes(y = `Solar Energy Consumption`, color = "Solar")) +
  geom_line(aes(y = `Wind Energy Consumption`, color = "Wind")) +
  scale_color_manual(
    name = "Energy Type",
    values = c(Solar = "red", Wind = "blue")
  ) +
  ggtitle("Solar and Wind Energy Consumption") +
  ylab("Energy Consumption") +
  xlab("Year") +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y")
```

Solar and Wind Energy Consumption



Decomposing the time series

The stats package has a function called `decompose()`. This function only take time series object. As the name says the `decompose` function will decompose your time series into three components: trend, seasonal and random. This is similar to what we did in the previous script, but in a more automated way. The random component is the time series without seasonal and trend component.

Additional info on `decompose()`.

- 1) You have two options: alternative and multiplicative. Multiplicative models exhibit a change in frequency over time.
- 2) The trend is not a straight line because it uses a moving average method to detect trend.
- 3) The seasonal component of the time series is found by subtracting the trend component from the original data then grouping the results by month and averaging them.
- 4) The random component, also referred to as the noise component, is composed of all the leftover signal which is not explained by the combination of the trend and seasonal component.

Q4

Transform wind and solar series into a time series object and apply the `decompose` function on them using the additive option, i.e., `decompose(ts_data, type = "additive")`. What can you say about the trend component? What about the random component? Does the random component look random? Or does it appear to still have some seasonality on it?

```

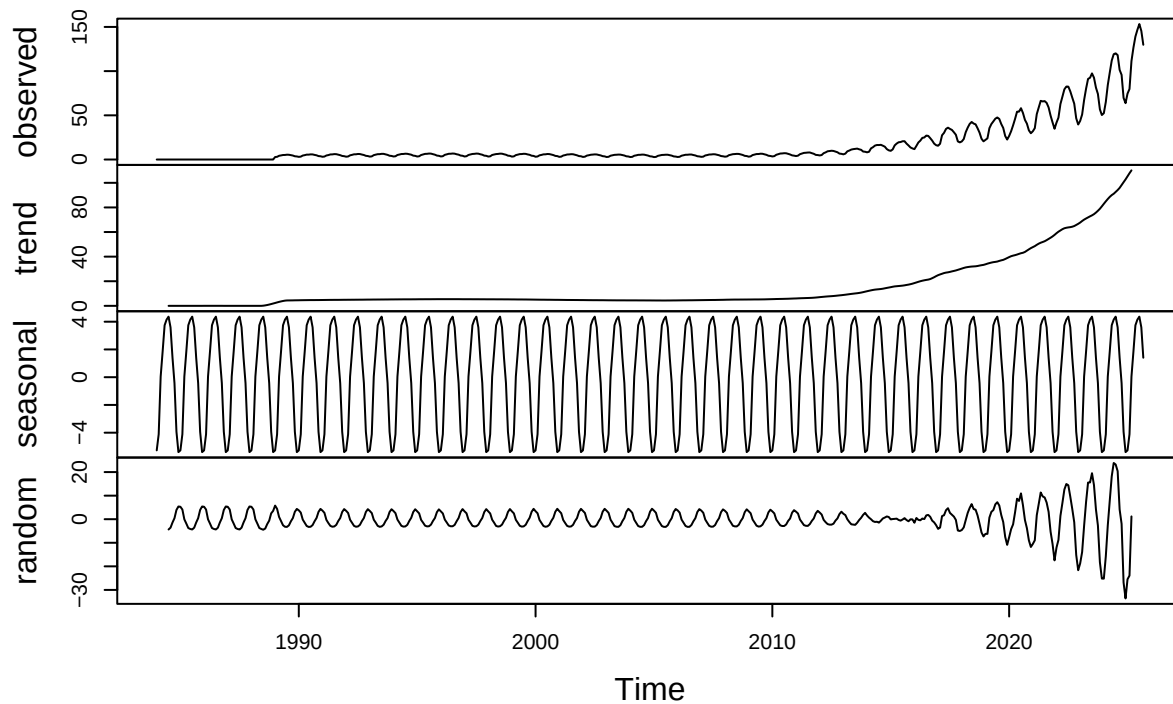
#Convert to time series
nobs <- nrow(SolarWind_data)

#Create vector t - time index
t <- 1:nobs
#transforming just the two columns of interest into ts object
ts_SolarWind_data <- ts(SolarWind_data[t,2:3], frequency=12,start=c(1984,1))
#frequency=12 because of monthly data. Data starts from Jan 1984.

#Decompose
decompose_Solar <- decompose(ts_SolarWind_data[,1],"additive")
plot(decompose_Solar)

```

Decomposition of additive time series

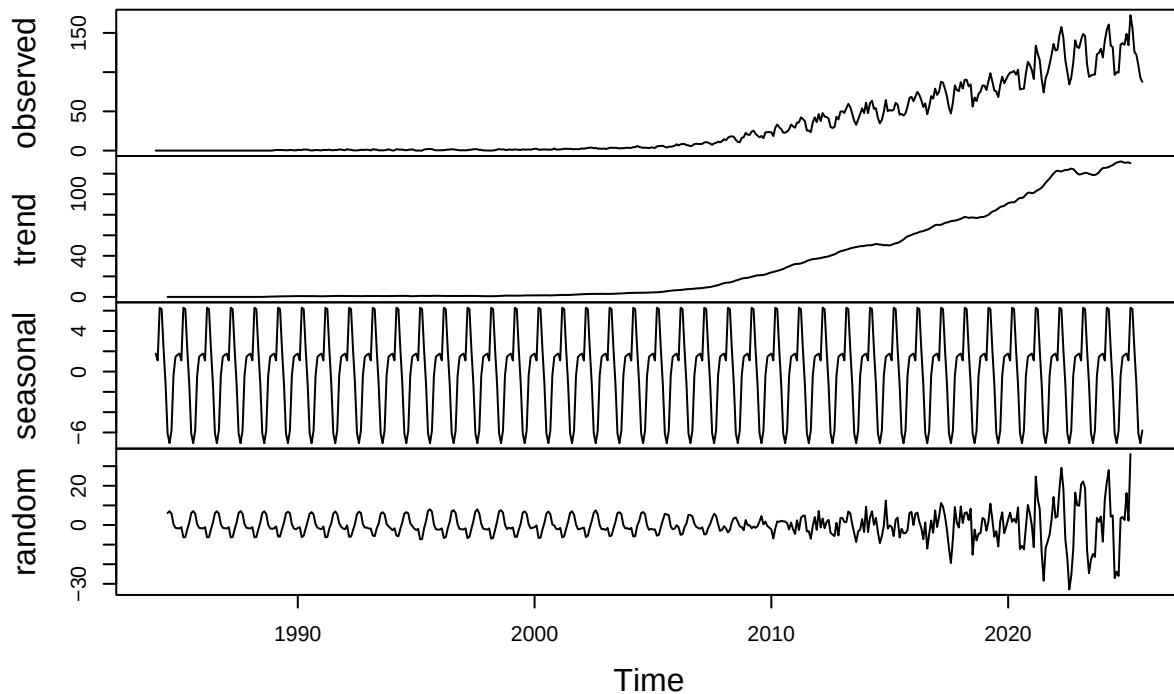


```

decompose_Wind <- decompose(ts_SolarWind_data[,2],"additive")
plot(decompose_Wind)

```

Decomposition of additive time series



#Inspect random component

```
solar_random <- decompose_Solar$random
```

```
mean_solar <- mean(na.exclude(solar_random)) #exclude NA or missing observation to compute mean and std
```

```
sd_solar <- sd(na.exclude(solar_random))
```

```
cat(mean_solar,sd_solar)
```

```
## -0.1432608 5.61712
```

```
wind_random <- decompose_Wind$random
```

```
mean_wind <- mean(na.exclude(wind_random)) #exclude NA or missing observation to compute mean and std
```

```
sd_wind <- sd(na.exclude(wind_random))
```

```
cat(mean_wind,sd_wind)
```

```
## 0.07840842 7.013719
```

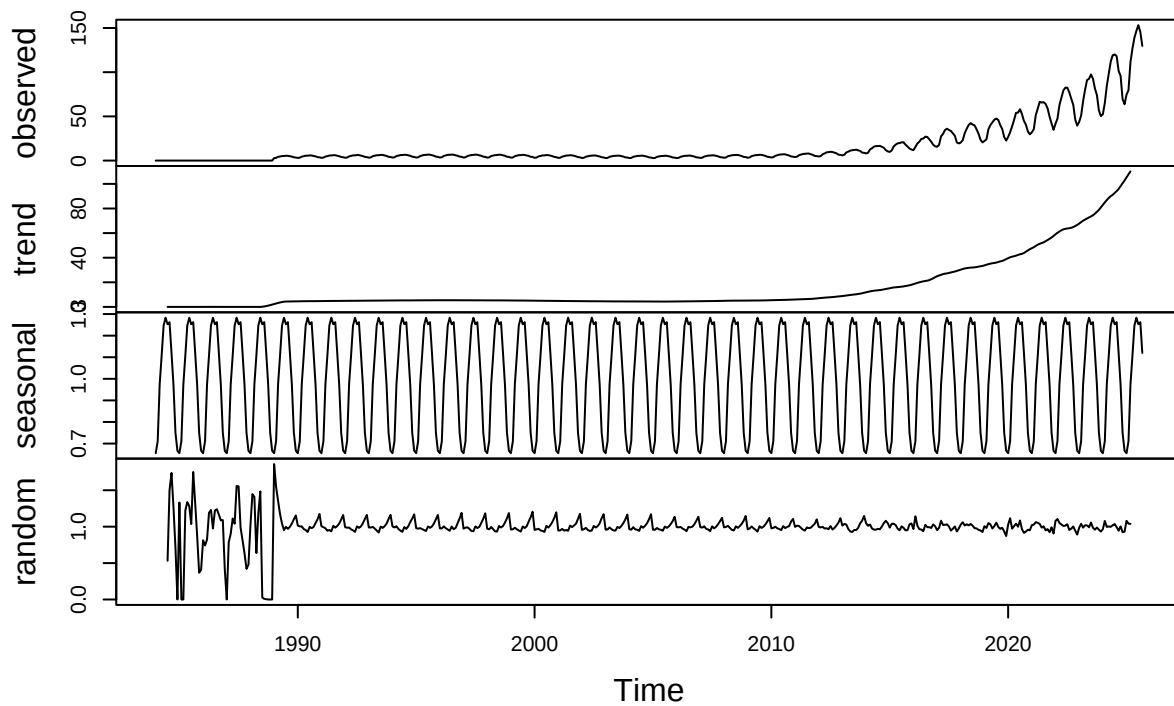
Answer: The trend component of solar energy shows a strong upward trend after year 2010, while its trend is flat before 2010. The trend component of wind energy shows a similar increasing trend: flat at the early stage and increases rapidly after 2008. The variances increase when the trends are moving upward. Both solar and wind's means are close to zero with a large standard deviation. The random components do not look that random, indicating that there is still seasonality on it.

Q5

Use the `decompose` function again but now change the type of the seasonal component from additive to multiplicative. What happened to the random component this time?

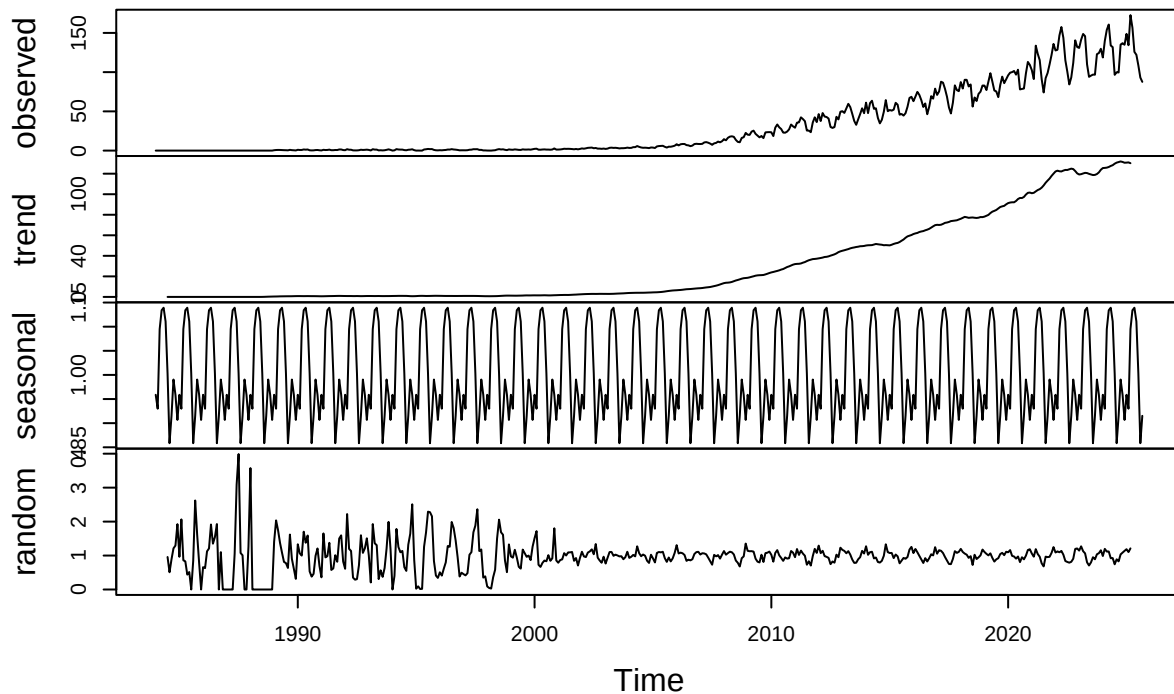
```
#Decompose  
decompose_Solar_new <- decompose(ts_SolarWind_data[,1],"multiplicative")  
plot(decompose_Solar_new)
```

Decomposition of multiplicative time series



```
decompose_Wind_new <- decompose(ts_SolarWind_data[,2],"multiplicative")  
plot(decompose_Wind_new)
```

Decomposition of multiplicative time series



> Answer: The random component for solar energy centers at 1, and the variances after year 1990 are relatively stable, indicating that seasonality and trend are captured. Similar to the solar energy plot, the random component of wind energy shows more stable fluctuations after early stage (2000). However, different than that of solar energy, the wind energy's random component's fluctuation is more irregular, and it does not center at 1 as solar energy does, indicating that wind energy is more variable than solar energy.

Q6

When fitting a model to this data, do you think you need all the historical data? Think about the data from 80s, 90s and early 20s. Are there any information from those years we might need to forecast the next six months of Solar and/or Wind consumption. Explain your response.

Answer: No. To forecast the next six months, recent data should be more effective than data from 80s and 90s. The data starts around 2010 can also be included since both wind and solar energy start to grow rapidly around 2010. Including data from many years ago when solar and wind developed slowly can lead to moving average distortion of the data we want to predict.

Q7

Create a new time series object where historical data starts on January 2014. Hint: use `filter()` function so that you don't need to point to row numbers, i.e, `filter(yyyy, year(Date) >= 2014)`. Apply the decompose function `type=additive` to this new time series. Comment on the results. Does the random component look random?

```

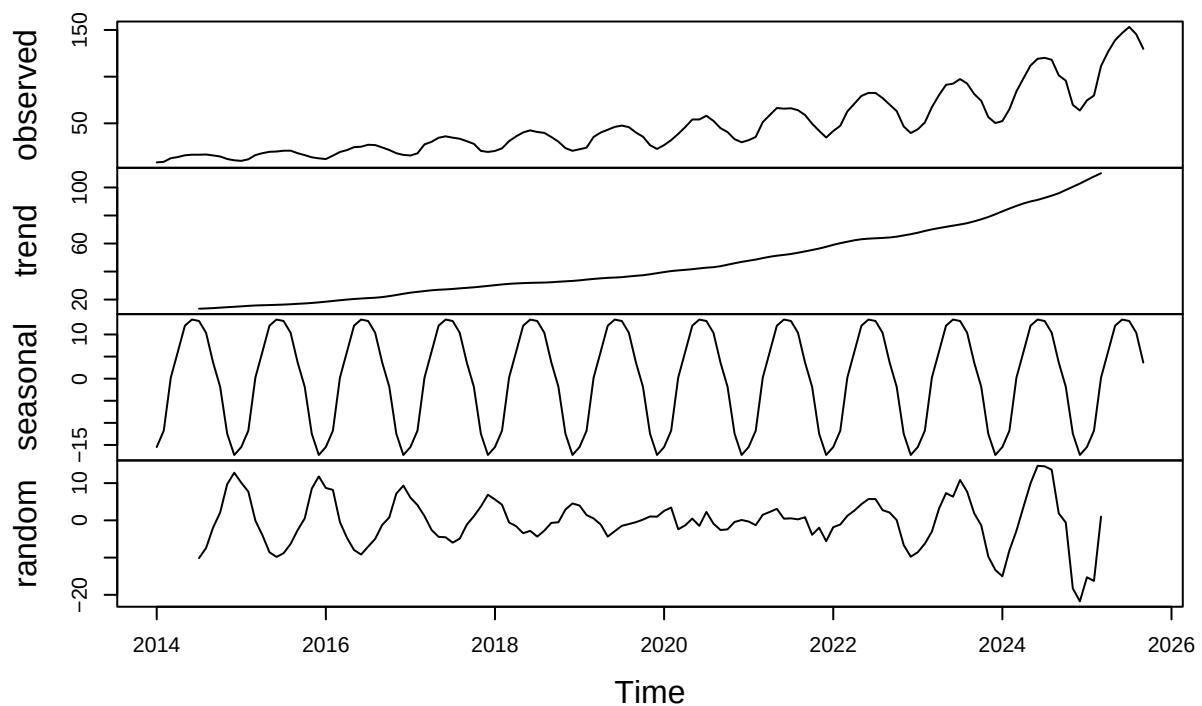
#Filter the data after Jan 2014
SW_2014 <- filter(SolarWind_data, year(Month) >= 2014)
#Convert to time series
nobs <- nrow(SW_2014)

#Create vector t - time index
t <- 1:nobs
#transforming just the two columns of interest into ts object
ts_SW_2014 <- ts(SW_2014[t,2:3], frequency=12,start=c(2014,1))

#Decompose
decompose_Solar_2014 <- decompose(ts_SW_2014[,1],"additive")
plot(decompose_Solar_2014)

```

Decomposition of additive time series

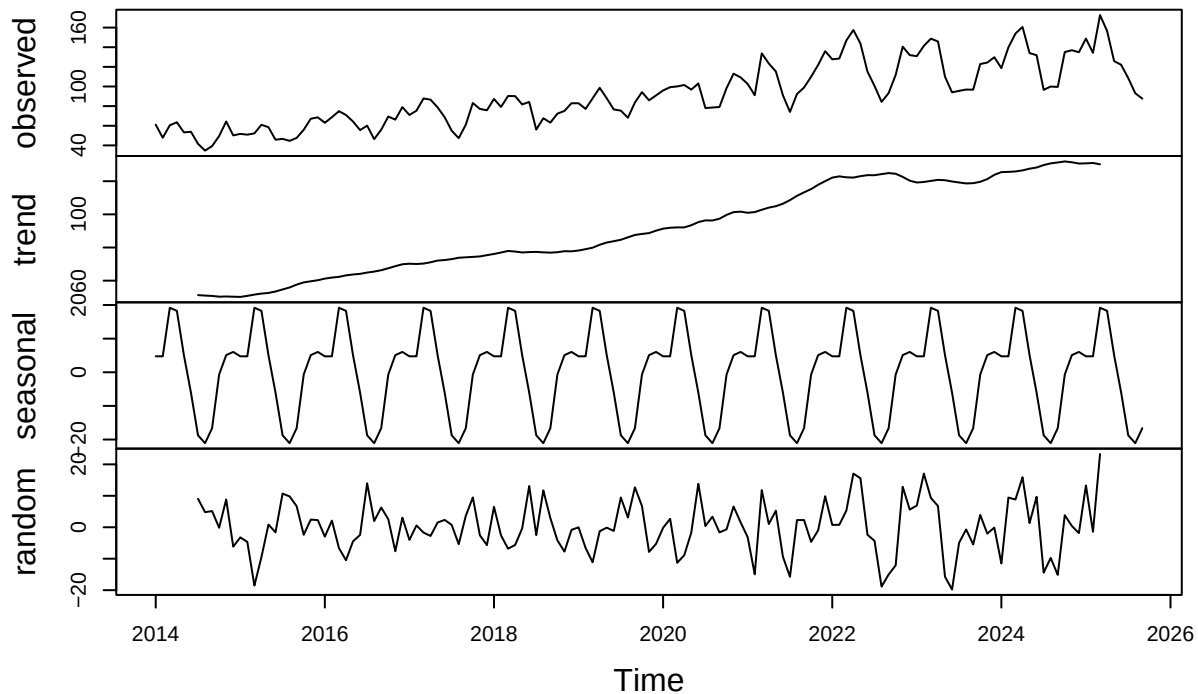


```

decompose_Wind_2014 <- decompose(ts_SW_2014[,2],"additive")
plot(decompose_Wind_2014)

```

Decomposition of additive time series



Answer: The random component does not look truly random. The solar energy's random component moves slow instead of being dense and move fast, and the random component decreases to -20 in 2024 and 2025. The wind energy's random component still shows seasonality with annual peaks throughout the series.

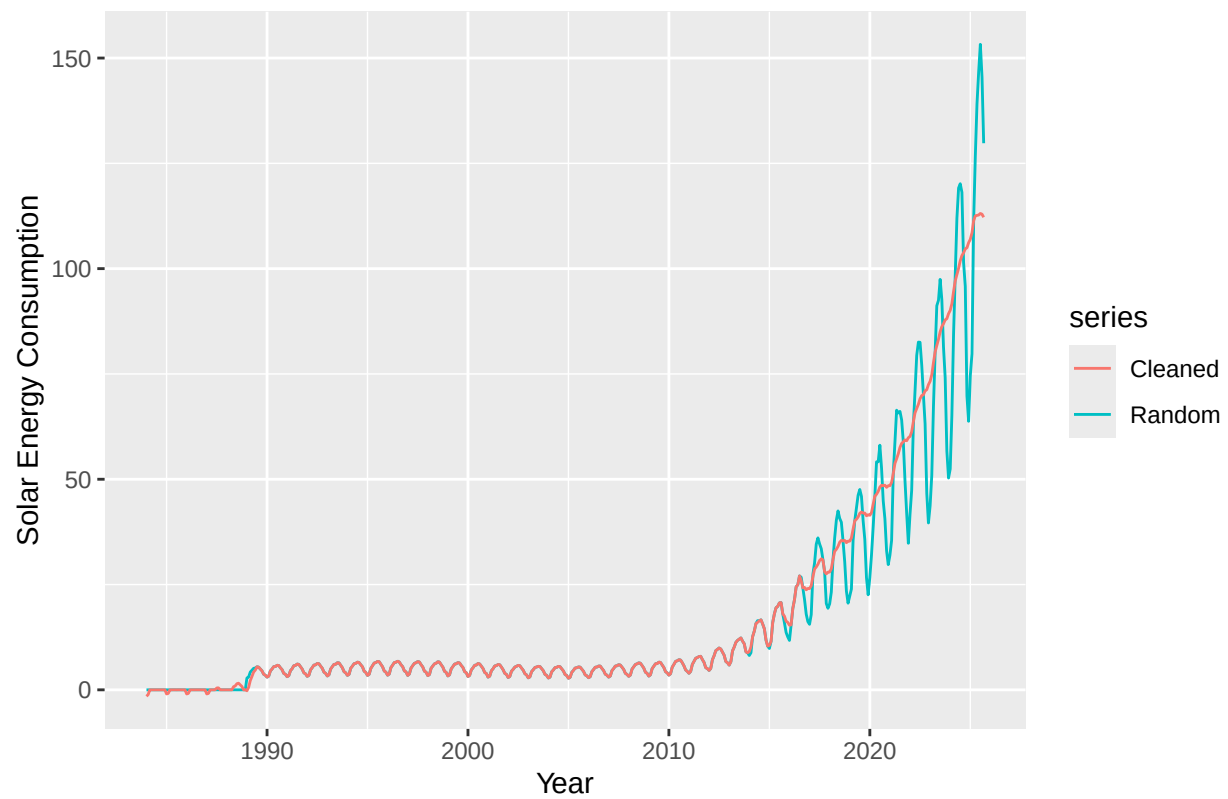
Identify and Remove outliers

Q8

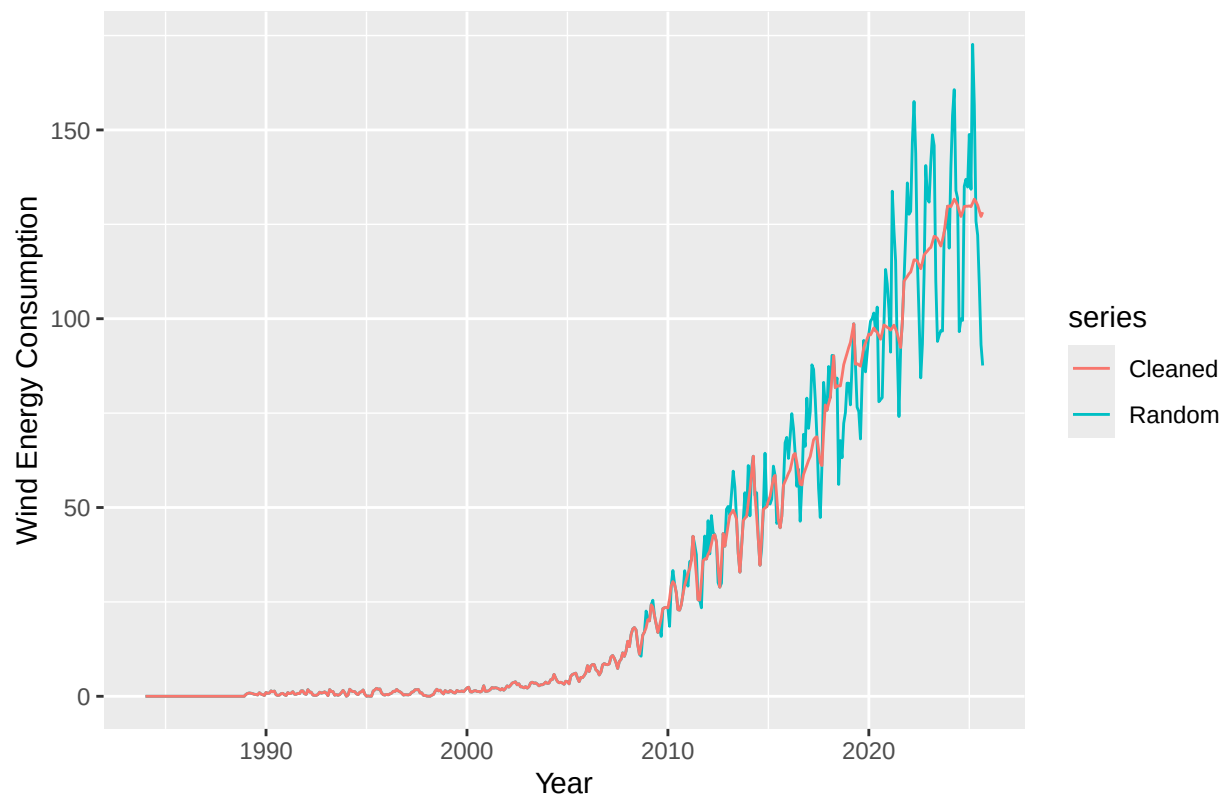
Apply the `tsclean()` to both time series object you created on Q4. Did the function removed any outliers from the series? Hint: Use `autoplot()` to check if there is difference between cleaned series and original series.

```
#Solar
clean_solar <- tsclean(ts_SolarWind_data[,1])

autoplot(ts_SolarWind_data[,1],series="Random")+
  autolayer(clean_solar,series="Cleaned")+
  xlab("Year")+
  ylab("Solar Energy Consumption")
```



```
#Wind  
clean_wind <- tsclean(ts_SolarWind_data[,2])  
autoplot(ts_SolarWind_data[,2],series="Random")+  
  autolayer(clean_wind,series="Cleaned")+  
  xlab("Year")+  
  ylab("Wind Energy Consumption")
```



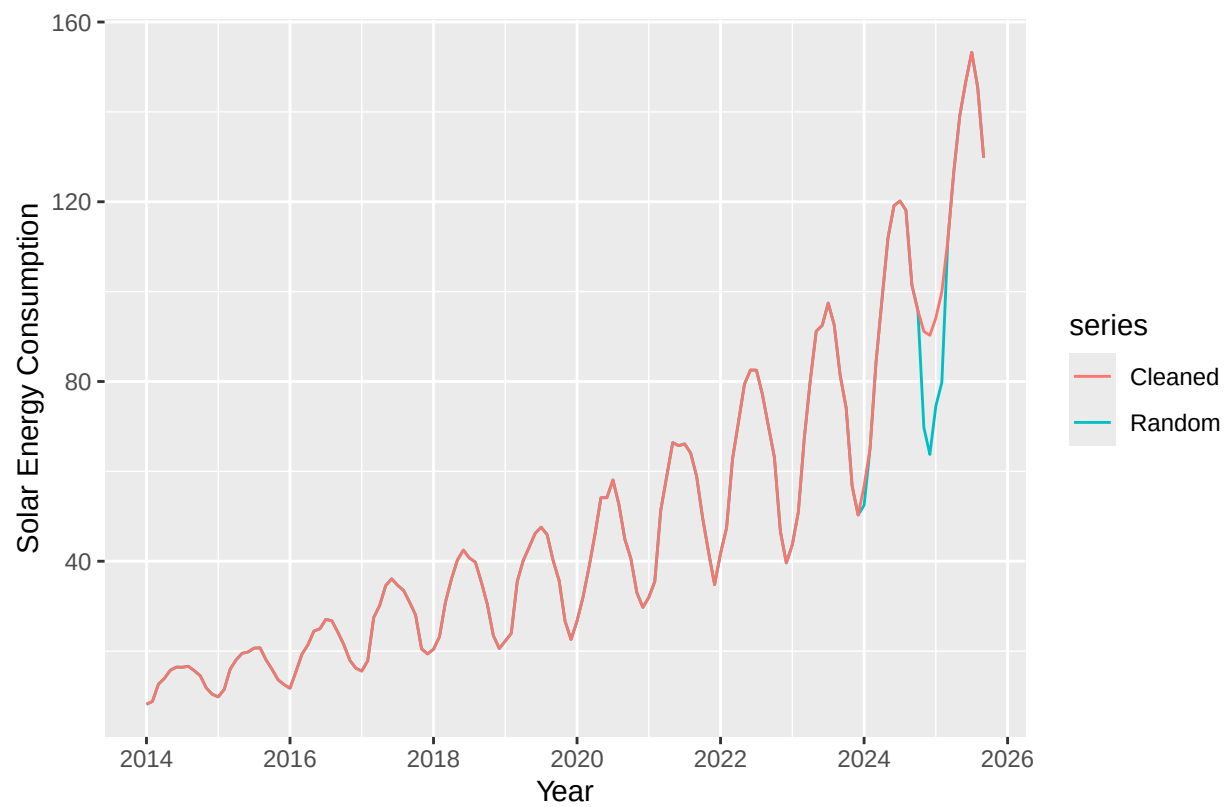
Yes. The function removed many outliers from the solar and wind series. After year 2010, when solar and wind energy start to grow rapidly, more outliers appear. The function helps remove such outliers.

Q9

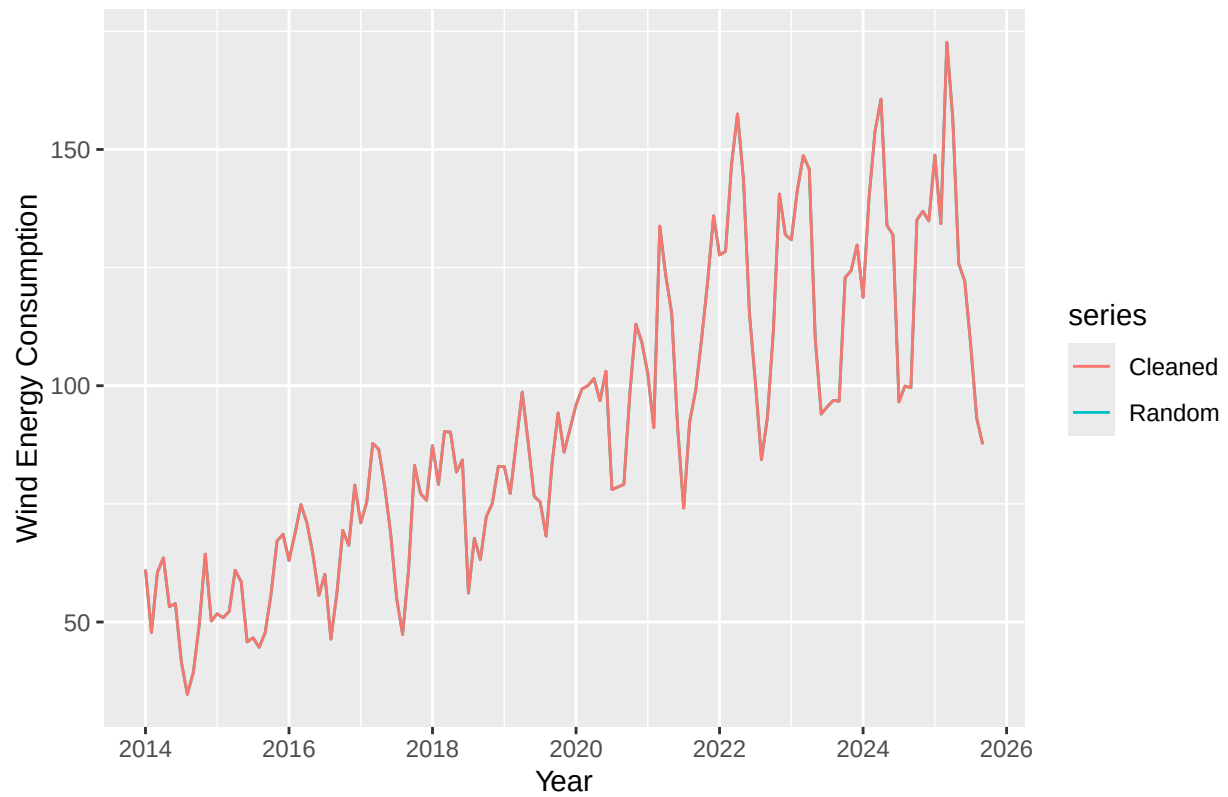
Redo number Q8 but now with the time series you created on Q7, i.e., the series starting in 2014. Using what `autoplot()` again what happened now? Did the function removed any outliers from the series?

```
#Solar
clean_solar_2014 <- tsclean(ts_SW_2014[,1])

autoplot(ts_SW_2014[,1],series="Random")+
  autolayer(clean_solar_2014,series="Cleaned")+
  xlab("Year")+
  ylab("Solar Energy Consumption")
```



```
#Wind
clean_wind_2014 <- tsclean(ts_SW_2014[,2])
autoplot(ts_SW_2014[,2],series="Random")+
  autolayer(clean_wind_2014,series="Cleaned")+
  xlab("Year")+
  ylab("Wind Energy Consumption")
```



Answer: The function helps remove a few outliers from the solar energy series and almost none from the wind energy series. This result shows that if we make the series start in 2014 (recent decade), we can eliminate many outliers than using data from 1980s.